User Manual

for S32K1_S32M24X PLATFORM Driver

Document Number: UM2PLATFORMASRR21-11 Rev0000R3.0.0 Rev. 1.0

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

This User Manual describes the NXP Semiconductor PLATFORM driver for S32K1_S32M24X. The PLATFORM driver configuration parameters and deviations from the specification are described in PLATFORM Driver chapter of this document. PLATFORM driver requirements and APIs are vendor-specific.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
IRQ	Interrupt request
LSB	Least Signifigant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	General Specification of Basic Software Modules	AUTOSAR Release R21-11
2	Specification of Communication Stack Types	AUTOSAR Release R21-11
3	Specification of Compiler Abstraction	AUTOSAR Release R21-11
4	Specification of Platform Types	AUTOSAR Release R21-11
5	Specification of Standard Types	AUTOSAR Release R21-11
6	S32K1xx Reference Manual	Rev.14.1, 01/2024
7	S32M24x Reference Manual	Rev. 5, 12/2024
8	S32K1xx Data Sheet	Rev. 14, 08/2021
9	S32M2xx Data Sheet	Rev. 7, 12/2024
10	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
11	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
12	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024

#	Title	Version
13	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
14	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
15	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
16	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
17	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
18	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

3.1 Requirements

PLATFORM is a complex driver, so there are no AUTOSAR requirements specific to this module. For the S32K1_↩ S32M24X platform, the PLATFORM module configures the interrupt controller, MCM, MPU. It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The PLATFORM driver configures platform specific settings, managing the interrupt requests and other system wide settings as defined in each hardware implementation.

Interrupt Controller

The configuration contains the list of interrupt requests, as defined per each platform. The application can enable the interrupts and set priorities. There is one Interrupt Controller belonging to each M0P/M4F core. The IsrHandler can be defined here or installed/updated at runtime via API.

Note

The handler installation works only if the interrupt vector table resides in RAM; the default implementation for startup/linker files provided by NXP enables this functionality.

Memory protection unit

MPU has 8 region descriptors (16 region descriptors for S32K148) configurable to protect memory (Flash, Ram, Peripheral memory) from access of masters (Core, Debugger, DMA, ENET). Each master can be authorized with 2 different access modes: user mode and supervisor mode.

Below you can find the descriptions for each file present in the Platform module:

File Name	File Type	Description
nvic.h	Stub file. Must be replaced by all integrators.	This file is a stub. This file include set priority grouping, enable , disable, set priority interrupt.
sys_init.h	Stub file. Must be replaced by all integrators.	this file is a stub. This file include some functions Function used to disable the interrupt number id. Function used to enable the interrupt number id and set up the priority. Function used to register the interrupt handler in the interrupt vectors. Function used to enable all interrupts. Function used to disable all interrupts. Function used to initiatialize clocks, system clock is system Pll 120 MHz. Function used to enter halt mode. Function used to enter stop mode. Function used to provide the CoreID to EUnit.
system.h	Stub file. Must be replaced by all integrators.	This file is a stub. This file include define some macros SCB Interrupt Control State Register Definitions.
nvic.c	Stub file. Must be replaced by all integrators.	This file is a stub. Set Priority Grouping. The function sets the priority grouping field using the required unlock sequence. The parameter PriorityGroup is assigned to the field SCB->AIRCR [10:8] PRIGROUP field.
sys_init.c	Stub file. Must be replaced by all integrators.	This file is a stub. This file include some functions need to initialize the clock.
system.c	Stub file. Must be replaced by all integrators.	This file is a stub. This file include some function. Function used to enter to supervisor mode. Check if it is needed to switch to supervisor mode and make the switch.
startup.c	Stub file. Must be replaced by all integrators.	This file is a stub. This file include sone functions necessary initializations for RAM. Copy the vector table from ROM to RAM. Copy initialized data from ROM to RAM. Copy code that should reside in RAM from ROM. Clear the zero-initialized data section.
exceptions.c	Stub file. Must be replaced by all integrators.	This file is a stub. This file necessary handler exception when running application.
startup_cm4.s	Stub file. Must be replaced by all integrators.	This file is a stub. This file needs to run before the initial application that is applied for the Cortex M4. Its start-up code shall initialize the base addresses for the interrupt and trap vector tables.

File Name	File Type	Description
startup_↩ cm0p.s	Stub file. Must be replaced by all integrators.	This file needs to run before the initial application that is applied for the Cortex M0+. Its start-up code shall initialize the base addresses for the interrupt and trap vector tables.
Vector_↩ Table.s	Stub file. Must be replaced by all integrators.	This file is a stub. This file necessary initializations for vector table.

Linker files

The provided linker file is a demo(reference) code, delivered for a single derivative (master part) common for all other derivatives.

- In order to benefit of the maximum available memory range, the integrators need to modify the linker file according to its specific derivatives.
- For example please refer to the Reference Manual attached memory map spreadsheet, change the memory section size in linker file according to the memory range for each derivatives.

3.3 Hardware Resources

#	Hardware IP	Description
1	S32 NVIC	ARM v6(S32K11X)/ARM v7(S32K14X & S32M24X) Nested Vectored Interrupt Controller
2	MPU	Memory Protection Unit
3	MCM	Miscellaneous Control Module

3.4 Deviations from Requirements

The driver deviates from the Platform Driver Software Specification in some places.

The table Status Column Description identifies the requirements that are not fully implemented, implemented differently, or out of scope for the Platform Driver.

The table Platform requirements deviations provides the "Status" column description.

Status Column Description

Term	Definition
N/S	Not In Scope
N/F	Not Fully Implemented
N/I	Not Implemented

Platform Requirements Deviations

Requirement	Status	Description	Notes
Platform_031	N/S	An enumeration, named Platform_GlobalStateType, shall expose the driver internal states: initialized/uninitialized.	Platform driver only implements registering and enabling/disabling interrupts, so this requirement Platform_031 is not necessary in platform driver. Please refer this ticket AAI-928 for details
Platform_049	N/S	A public function, named Platform_Mpu_M7_MulticoreInit(), shall initialize the Memory Protection Unit for the core M7. Platform_Mpu_M7_MulticoreInit shall have the following properties: * Service name: Platform_Mpu_M7_MulticoreInit * Syntax: void Platform_Mpu_M7_MulticoreInit(void) * Service ID[hex]: 0x3D * Sync/Async: Synchronous * Reentrancy: Non-Reentrant * Parameters(in): None * Parameters(inout): None * Parameters(out): None * Return value: void * Description: Initializes the Memory Protection Unit general parameters and region configurations for the core which is running.	This function is not needed on platform driver, because Platform_init will initialize the Memory Protection Unit general parameters and region configurations for the core which is running.
CPR_RTD_00420.platform	N/F	If DET error reporting is enabled, the PLATFORM will check upon each API call if the requested resource is configured to be available on the current partition, and in case of error will return PLATFORM_E_PARAM_CONFIG.	The Platform driver only checks upon the Platform_Init API call if the requested resource is configured to be available on the current partition. This check does not exist for the remaining APIs of the Platform driver.

3.5 Driver Limitations

The PLATFORM driver software have some following limitations for RTD S32K1 :

- Only one precompile configuration variant supported in the configuration tool.

3.6 Driver usage and configuration tips

In multi-partition use case:

- The MPU should be initialized by only one partition if multiple partitions are associated with the same core.
- The interrupt handler for each core must be the same if a common interrupt vector table is used across all those cores.

3.7 Runtime errors

The driver does not generate DEM errors.

The development errors generated through DET are:

Error Code	Condition triggering the error
PLATFORM_E_PARAM_POINTER	Invalid pointer (null pointer) for parameters passed as reference
PLATFORM_E_PARAM_OUT_OF_RANGE	Parameter out of range
PLATFORM_E_VECTOR_TABLE_READ_ONLY	vector table resides in target flash
PLATFORM_E_PARAM_CONFIG	call from wrong mapped partition

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: *#define <Mip>Conf_<Container_ShortName>_<Container_ID>*

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

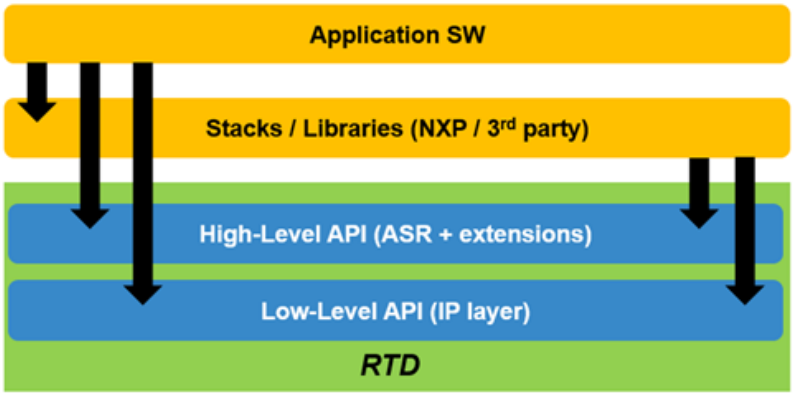

Figure 3.1 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools

- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
  1. Initialization of the device clock tree so other periphery can be initialized
  2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
  3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
  4. Initialization of specific drivers
  5. Runtime functionality
  6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

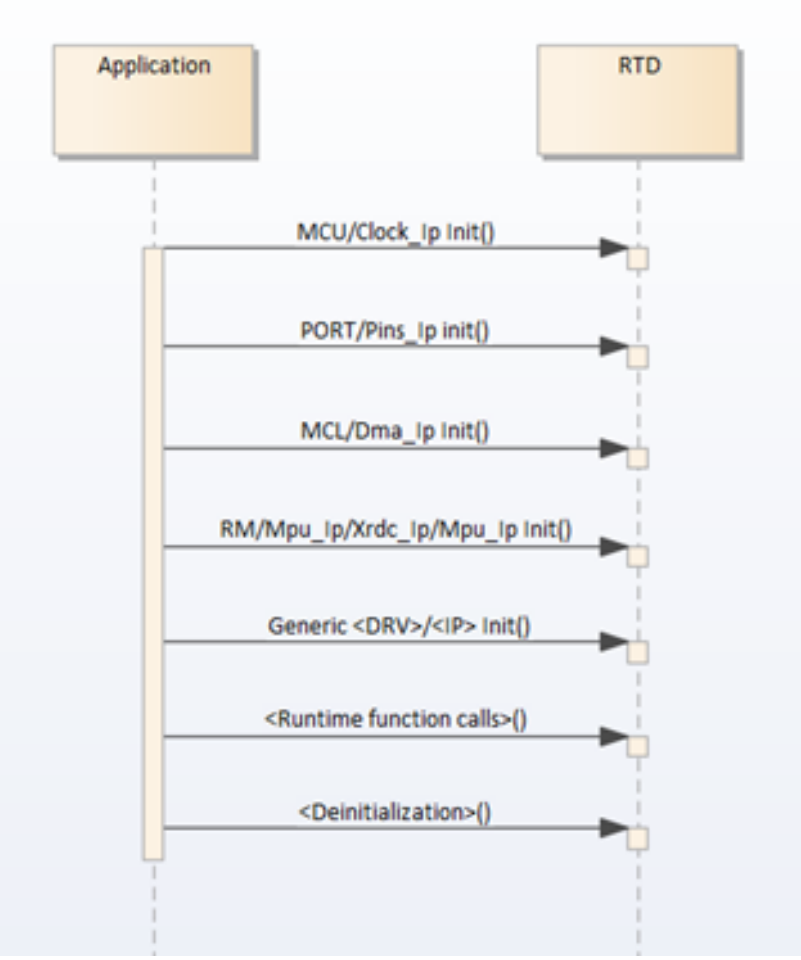

Figure 3.2 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

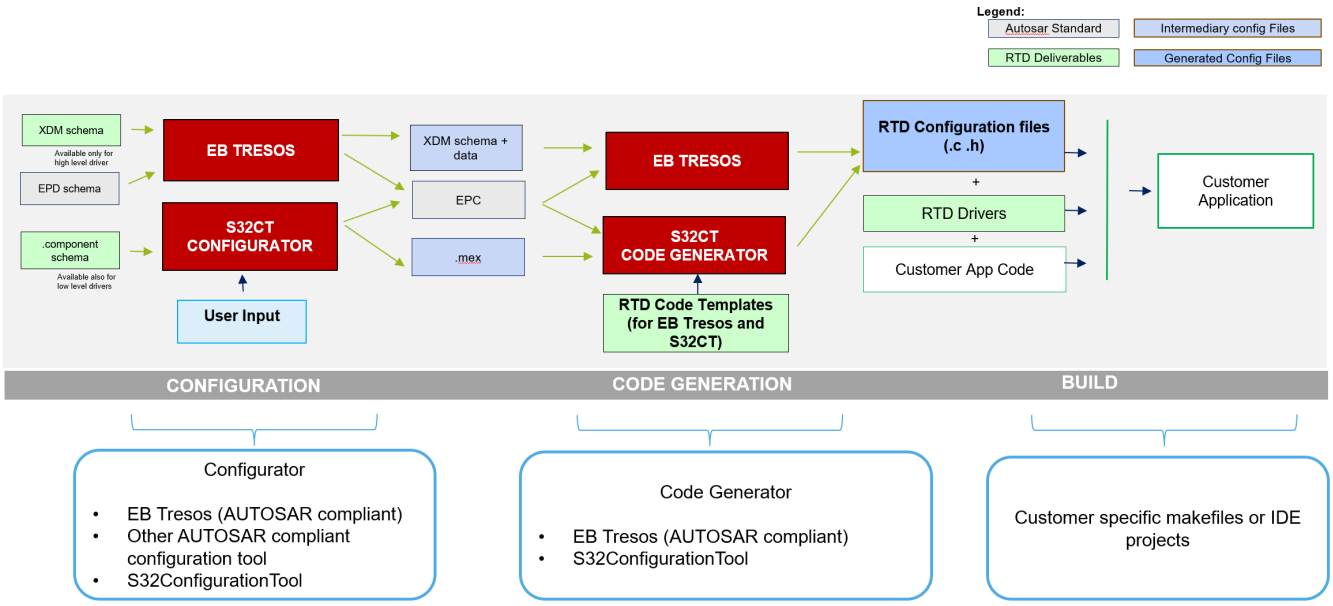

Figure 4.1 Configuration generation flow

- Platform
- INTCTRL_IP
- SYSTEM_IP
- IPV_MPU_IP

4.1 Platform

- Module Platform
  - Container GeneralConfiguration
    - Parameter PlatformDevErrorDetect
    - Parameter PlatformMcmConfigurable
    - Parameter PlatformMpuConfigurable
    - Parameter PlatformIpAPIsAvailable
    - Parameter PlatformEnableUserModeSupport
    - Parameter PlatformMulticoreSupport
    - Parameter PlatformEnableVtorConfiguration
    - Parameter PlatformEnableIntCtrlConfiguration
    - Reference PlatformEcucPartitionRef
  - Container McmConfig
    - Parameter SystemAhbSlavePrio
    - Reference PlatformMcmEcucPartitionRef
    - Container SystemIsrConfig
      - Parameter SystemIsrName
      - Parameter SystemIsrEnabled
  - Container Mpu_Configuration
    - Container MpuRegionConfig
      - Parameter RegionNumber
      - Parameter StartAddress
      - Parameter EndAddress
      - Parameter ProcessIdentifierEnableMaster0
      - Parameter SupervisorModeAccessBusMaster0
      - Parameter UserModeAccessBusMaster0
      - Parameter ProcessIdentifierEnableMaster1
      - Parameter SupervisorModeAccessBusMaster1
      - Parameter UserModeAccessBusMaster1
      - Parameter SupervisorModeAccessBusMaster2
      - Parameter UserModeAccessBusMaster2
      - Parameter SupervisorModeAccessBusMaster3
      - Parameter UserModeAccessBusMaster3
      - Parameter ProcessIdentifier
      - Parameter ProcessIdentifierMask
      - Reference PlatformMpuEcucPartitionRef

- Container IntCtrlConfig
  - Parameter PlatformVtorAddressConfig
  - Reference PlatformNvicEcucPartitionRef
  - Container PlatformIsrConfig
    - Parameter IsrName
    - Parameter IsrEnabled
    - Parameter IsrPriority
    - Parameter IsrHandler
- Container CommonPublishedInformation
  - Parameter ArReleaseMajorVersion
  - Parameter ArReleaseMinorVersion
  - Parameter ArReleaseRevisionVersion
  - Parameter ModuleId
  - Parameter SwMajorVersion
  - Parameter SwMinorVersion
  - Parameter SwPatchVersion
  - Parameter VendorApiInfix
  - Parameter VendorId

4.1.1 Module Platform

Configuration of Platform module.

Included containers:

- GeneralConfiguration
- McmConfig
- Mpu_Configuration
- IntCtrlConfig
- CommonPublishedInformation

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	False
supportedConfigVariants	VARIANT-PRE-COMPILE, VARIANT-POST-BUILD

4.1.2 Container GeneralConfiguration

GeneralConfiguration

This container contains the global configuration parameters of the Non-Autosar Platform driver.

Note: Implementation Specific Parameter.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter PlatformDevErrorDetect

PlatformDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.1.4 Parameter PlatformMcmConfigurable

PlatformMcmConfigurable

Check this in order to be able to configure Miscellaneous Control settings.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.5 Parameter PlatformMpuConfigurable

PlatformMpuConfigurable

Check this in order to be able to use the MPU.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.6 Parameter PlatformIpAPIsAvailable

PlatformIpAPIsAvailable

Enable or disable IP layer APIs which are not used by APIs at High Level Driver (HLD). Following APIs are affected:

IntCtrl_Ip_SetPending

IntCtrl_Ip_GetPending

IntCtrl_Ip_GetActive

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.7 Parameter PlatformEnableUserModeSupport

When this parameter is enabled, the Platform module will adapt to run from User Mode, with the following measures:

b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.8 Parameter PlatformMulticoreSupport

This parameter globally enables the possibility to support multi-partition. If this parmeter is enabled, at least one EcucPartition needs to be defined (in all variants).

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.9 Parameter PlatformEnableVtorConfiguration

When this parameter is enabled, the Platform module will allow the user the manually configure the Vector Table Offset Register address:

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.10 Parameter PlatformEnableIntCtrlConfiguration

When this parameter is enabled, the Platform module will allow to configure for Interrupt Controller module

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.1.11 Reference PlatformEcucPartitionRef

Maps the Platform driver to zero a multiple ECUC partitions to make the modules API available in this partition.

Note: Each PlatformEcucPartitionRef should map to a M7 core, one by one

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0

Property	Value
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.12 Container McmConfig

Vendor specific:

Miscellaneous Control Configuration

Included subcontainers:

- SystemIsrConfig

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.13 Parameter SystemAhbSlavePrio

Vendor specific:

Configures the access priority on the AHBS port of the Cortex-M7.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	Round_robin
literals	['Round_robin', 'AHB_Slave_priority']

4.1.14 Reference PlatformMcmEcucPartitionRef

Maps a instance of Mcm to ECUC partitions.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.15 Container SystemIsrConfig

Vendor specific:

Configuration for core-related interrupts.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	6
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.16 Parameter SystemIsrName

Vendor specific:

Interrupt Name.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FPU_INPUT_DENORMAL_IRQ
literals	['FPU_INPUT_DENORMAL_IRQ', 'FPU_INEXACT_IRQ', 'FPU_UNDE↩ RFLOW_IRQ', 'FPU_OVERFLOW_IRQ', 'FPU_DIVIDE_BY_ZERO_IRQ', 'FPU_INVALID_OPERATION_IRQ']

4.1.17 Parameter SystemIsrEnabled

Vendor specific: Switch to indicate if the interrupt is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.18 Container Mpu_Configuration

Configuration for the MPU module.

Included subcontainers:

- MpuRegionConfig

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.19 Container MpuRegionConfig

Vendor specific:

Configuration for Mpu regions

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.20 Parameter RegionNumber

Vendor specific:

Hardware Region Number to be configuration.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	15
min	0

4.1.21 Parameter StartAddress

Vendor specific:

Start Address of the region.

Note: For RegionNumber 0, Start address always is 0x00000000, This field will be ignored.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	4294967264
min	0

4.1.22 Parameter EndAddress

Vendor specific:

End Address of the region.

Note: For RegionNumber 0, End address always is 0xFFFFFFFF, This field will be ignored.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	31
max	4294967295
min	31

4.1.23 Parameter ProcessIdentifierEnableMaster0

Bus Master 0 (Core) Process Identifier enable.

True: Include the process identifier and mask in the region hit evaluation.

False: Do not include the process identifier in the evaluation.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.24 Parameter SupervisorModeAccessBusMaster0

Vendor specific: Defines the access controls for bus master 0 in Supervisor mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MPU_SUPERVISOR_MODE_RWX
literals	['MPU_SUPERVISOR_MODE_RWX', 'MPU_SUPERVISOR_MODE_RX', 'MPU_SUPERVISOR_MODE_RW', 'MPU_SUPERVISOR_MODE_AS_U↩SER_MODE']

4.1.25 Parameter UserModeAccessBusMaster0

Vendor specific: Defines the access controls for bus master 0 in User mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MPU_USER_MODE_NONE
literals	['MPU_USER_MODE_NONE', 'MPU_USER_MODE_RWX', 'MPU_USE↩R_MODE_RW', 'MPU_USER_MODE_RX', 'MPU_USER_MODE_R', 'M↩PU_USER_MODE_WX', 'MPU_USER_MODE_W', 'MPU_USER_MOD↩E_X']

4.1.26 Parameter ProcessIdentifierEnableMaster1

Bus Master 1 (Debugger) Process Identifier enable.

True: Include the process identifier and mask in the region hit evaluation.

False: Do not include the process identifier in the evaluation.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.27 Parameter SupervisorModeAccessBusMaster1

Vendor specific: Defines the access controls for bus master 1 in Supervisor mode.

Note: Can not set supervisor mode access bus master 1 for region 0

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MPU_SUPERVISOR_MODE_RWX
literals	['MPU_SUPERVISOR_MODE_RWX', 'MPU_SUPERVISOR_MODE_RX', 'MPU_SUPERVISOR_MODE_RW', 'MPU_SUPERVISOR_MODE_AS_U↩SER_MODE']

4.1.28 Parameter UserModeAccessBusMaster1

Vendor specific: Defines the access controls for bus master 1 in User mode.

Note: Can not set user mode access bus master 1 for region 0

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MPU_USER_MODE_NONE
literals	['MPU_USER_MODE_NONE', 'MPU_USER_MODE_RWX', 'MPU_USE↩ R_MODE_RW', 'MPU_USER_MODE_RX', 'MPU_USER_MODE_R', 'M↩ PU_USER_MODE_WX', 'MPU_USER_MODE_W', 'MPU_USER_MOD↩ E_X']

4.1.29 Parameter SupervisorModeAccessBusMaster2

Vendor specific: Defines the access controls for bus master 2 in Supervisor mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MPU_SUPERVISOR_MODE_RWX
literals	['MPU_SUPERVISOR_MODE_RWX', 'MPU_SUPERVISOR_MODE_RX', 'MPU_SUPERVISOR_MODE_RW', 'MPU_SUPERVISOR_MODE_AS_U↩ SER_MODE']

4.1.30 Parameter UserModeAccessBusMaster2

Vendor specific: Defines the access controls for bus master 2 in User mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MPU_USER_MODE_NONE
literals	['MPU_USER_MODE_NONE', 'MPU_USER_MODE_RWX', 'MPU_USE↩R_MODE_RW', 'MPU_USER_MODE_RX', 'MPU_USER_MODE_R', 'M↩PU_USER_MODE_WX', 'MPU_USER_MODE_W', 'MPU_USER_MOD↩E_X']

4.1.31 Parameter SupervisorModeAccessBusMaster3

Vendor specific: Defines the access controls for bus master 3 in Supervisor mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MPU_SUPERVISOR_MODE_RWX
literals	['MPU_SUPERVISOR_MODE_RWX', 'MPU_SUPERVISOR_MODE_RX', 'MPU_SUPERVISOR_MODE_RW', 'MPU_SUPERVISOR_MODE_AS_U↩SER_MODE']

4.1.32 Parameter UserModeAccessBusMaster3

Vendor specific: Defines the access controls for bus master 3 in User mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MPU_USER_MODE_NONE
literals	['MPU_USER_MODE_NONE', 'MPU_USER_MODE_RWX', 'MPU_USE↩ R_MODE_RW', 'MPU_USER_MODE_RX', 'MPU_USER_MODE_R', 'M↩ PU_USER_MODE_WX', 'MPU_USER_MODE_W', 'MPU_USER_MOD↩ E_X']

4.1.33 Parameter ProcessIdentifier

Vendor specific: Specifies the process identifier that is included in the region hit determination

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	255
min	0

4.1.34 Parameter ProcessIdentifierMask

Vendor specific: Provides a masking capability so that multiple process identifiers

can be included as part of the region hit determination

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	255
min	0

4.1.35 Reference PlatformMpuEcucPartitionRef

Maps a instance of Mpu to ECUC partitions.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.36 Container IntCtrlConfig

Configuration for the interrupts.

Included subcontainers:

- PlatformIsrConfig

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.37 Parameter PlatformVtorAddressConfig

Configure the address where the Interrupt Vector starts

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.38 Reference PlatformNvicEcucPartitionRef

Maps an instance of Nvic ECUC partitions.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.39 Container PlatformIsrConfig

Vendor specific:

Configuration for interrupt requests.

Warning: This is a precompile configuration.If you uncheck a ISR, you will not be able to enable the respective channel or error functionality at post build time.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	128

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.40 Parameter IsrName

Vendor specific:

Interrupt Name.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	DMA0_IRQn

Property	Value
literals	['DMA0_IRQn', 'DMA1_IRQn', 'DMA2_IRQn', 'DMA3_IRQn', 'DMA4_IRQn', 'DMA5_IRQn', 'DMA6_IRQn', 'DMA7_IRQn', 'DMA8_IRQn', 'DMA9_IRQn', 'DMA10_IRQn', 'DMA11_IRQn', 'DMA12_IRQn', 'DMA13_IRQn', 'DMA14_IRQn', 'DMA15_IRQn', 'DMA_Error_IRQn', 'MCM_IRQn', 'FTFC_CMD_IRQn', 'FTFC_Read_Collision_IRQn', 'LVD_LVW_IRQn', 'FTFC_Fault_IRQn', 'WDOG_EWM_IRQn', 'RCM_IRQn', 'LPI2C0_Master_IRQn', 'LPI2C0_Slave_IRQn', 'LPSPI0_IRQn', 'LPSPI1_IRQn', 'LPSPI2_IRQn', 'LPI2C1_Master_IRQn', 'LPI2C1_Slave_IRQn', 'LPUART0_RxTx_IRQn', 'LPUART1_RxTx_IRQn', 'LPUART2_RxTx_IRQn', 'ADC0_IRQn', 'ADC1_IRQn', 'CMP0_IRQn', 'ERM_single_fault_IRQn', 'ERM_double_fault_IRQn', 'RTC_IRQn', 'RTC_Seconds_IRQn', 'LPIT0_Ch0_IRQn', 'LPIT0_Ch1_IRQn', 'LPIT0_Ch2_IRQn', 'LPIT0_Ch3_IRQn', 'PDB0_IRQn', 'SAI1_TX_SYNC_IRQn', 'SAI1_RX_SYNC_IRQn', 'SCG_IRQn', 'LPTMR0_IRQn', 'PORTA_IRQn', 'PORTB_IRQn', 'PORTC_IRQn', 'PORTD_IRQn', 'PORTE_IRQn', 'SWI_IRQn', 'QSPI_Ored_IRQn', 'PDB1_IRQn', 'FLEXIO_IRQn', 'SAI0_TX_SYNC_IRQn', 'SAI0_RX_SYNC_IRQn', 'ENET_Timer_IRQn', 'ENET_TX_Buffer_IRQn', 'ENET_RX_Buffer_IRQn', 'ENET_PRE_IRQn', 'ENET_STOP_IRQn', 'ENET_WAKE_IRQn', 'CAN0_ORed_IRQn', 'CAN0_Error_IRQn', 'CAN0_Wake_Up_IRQn', 'CAN0_ORed_0_15_MB_IRQn', 'CAN0_ORed_16_31_MB_IRQn', 'CAN1_ORed_IRQn', 'CAN1_Error_IRQn', 'CAN1_ORed_0_15_MB_IRQn', 'CAN1_ORed_16_31_MB_IRQn', 'CAN2_ORed_IRQn', 'CAN2_Error_IRQn', 'CAN2_ORed_0_15_MB_IRQn', 'CAN2_ORed_16_31_MB_IRQn', 'FTM0_Ch0_Ch1_IRQn', 'FTM0_Ch2_Ch3_IRQn', 'FTM0_Ch4_Ch5_IRQn', 'FTM0_Ch6_Ch7_IRQn', 'FTM0_Fault_IRQn', 'FTM0_Ovf_Reload_IRQn', 'FTM1_Ch0_Ch1_IRQn', 'FTM1_Ch2_Ch3_IRQn', 'FTM1_Ch4_Ch5_IRQn', 'FTM1_Ch6_Ch7_IRQn', 'FTM1_Fault_IRQn', 'FTM1_Ovf_Reload_IRQn', 'FTM2_Ch0_Ch1_IRQn', 'FTM2_Ch2_Ch3_IRQn', 'FTM2_Ch4_Ch5_IRQn', 'FTM2_Ch6_Ch7_IRQn', 'FTM2_Fault_IRQn', 'FTM2_Ovf_Reload_IRQn', 'FTM3_Ch0_Ch1_IRQn', 'FTM3_Ch2_Ch3_IRQn', 'FTM3_Ch4_Ch5_IRQn', 'FTM3_Ch6_Ch7_IRQn', 'FTM3_Fault_IRQn', 'FTM3_Ovf_Reload_IRQn', 'FTM4_Ch0_Ch1_IRQn', 'FTM4_Ch2_Ch3_IRQn', 'FTM4_Ch4_Ch5_IRQn', 'FTM4_Ch6_Ch7_IRQn', 'FTM4_Fault_IRQn', 'FTM4_Ovf_Reload_IRQn', 'FTM5_Ch0_Ch1_IRQn', 'FTM5_Ch2_Ch3_IRQn', 'FTM5_Ch4_Ch5_IRQn', 'FTM5_Ch6_Ch7_IRQn', 'FTM5_Fault_IRQn', 'FTM5_Ovf_Reload_IRQn', 'FTM6_Ch0_Ch1_IRQn', 'FTM6_Ch2_Ch3_IRQn', 'FTM6_Ch4_Ch5_IRQn', 'FTM6_Ch6_Ch7_IRQn', 'FTM6_Fault_IRQn', 'FTM6_Ovf_Reload_IRQn', 'FTM7_Ch0_Ch1_IRQn', 'FTM7_Ch2_Ch3_IRQn', 'FTM7_Ch4_Ch5_IRQn', 'FTM7_Ch6_Ch7_IRQn', 'FTM7_Fault_IRQn', 'FTM7_Ovf_Reload_IRQn']

4.1.41 Parameter IsrEnabled

Vendor specific: Switch to indicate if the interrupt is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.42 Parameter IsrPriority

Priority of the interrupt

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.43 Parameter IsrHandler

Function to be installed as the interrupt handler.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	undefined_handler

4.1.44 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.45 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	4
max	4
min	4

4.1.46 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	7
max	7
min	7

4.1.47 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0
min	0

4.1.48 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	255
max	255
min	255

4.1.49 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	3
max	3
min	3

4.1.50 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0
min	0

4.1.51 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0
min	0

4.1.52 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	

4.1.53 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	43
max	43
min	43

4.2 INTCTRL_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module IntCtrl_Ip
  - Container IntCtrlConfigGeneral
    - Parameter IntCtrlDevErrorDetect
    - Parameter PlatformEnableUserModeSupport
  - Container IntCtrlConfig

4.2.1 Module IntCtrl_Ip

IntCtrl_Ip configuration

Included containers:

- IntCtrlConfigGeneral
- IntCtrlConfig

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-PRE-COMPILE;VARIANT-POST-BUILD

4.2.2 Container IntCtrlConfigGeneral

* Configuration for the interrupts at controller level (core specific). *

Included subcontainers:

- None

Property	Value
type	CONTAINER
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.2.3 Parameter IntCtrlDevErrorDetect

Vendor specific:

Enables the development error detection feature.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	False
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.3 SYSTEM_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module System_Ip
  - Container McmConfig
    - Parameter SystemDevErrorDetect
    - Parameter PlatformEnableUserModeSupport

4.3.1 Module System_Ip

System_Ip configuration

Included containers:

- McmConfig

Property	Value
type	MODULE-DEF

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-PRE-COMPILE;VARIANT-POST-BUILD

4.3.2 Parameter SystemDevErrorDetect

Enables the development error detection feature.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	False
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.4 IPV_MPU_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module IPV_Mpu_Ip
  - Container MPU_GeneralConfiguration
    - Parameter MPU_DevErrorDetect
    - Parameter MPU_UserModeSupport
  - Container Mpu_Configuration
    - Container MpuRegionConfig
      - Parameter RegionNumber
      - Parameter StartAddress
      - Parameter EndAddress
      - Parameter ProcessIdentifierEnableMaster0
      - Parameter SupervisorModeAccessBusMaster0
      - Parameter UserModeAccessBusMaster0
      - Parameter ProcessIdentifierEnableMaster1

- Parameter SupervisorModeAccessBusMaster1
- Parameter UserModeAccessBusMaster1
- Parameter SupervisorModeAccessBusMaster2
- Parameter UserModeAccessBusMaster2
- Parameter SupervisorModeAccessBusMaster3
- Parameter UserModeAccessBusMaster3
- Parameter ProcessIdentifier
- Parameter ProcessIdentifierMask

4.4.1 Module IPV_Mpu_Ip

IPV_Mpu_Ip configuration

Included containers:

- MPU_GeneralConfiguration
- Mpu_Configuration

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-PRE-COMPILE;VARIANT-POST-BUILD

4.4.2 Container MPU_GeneralConfiguration

N/A

Included subcontainers:

- None

Property	Value
type	CONTAINER
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.4.3 Parameter MPU_DevErrorDetect

Enables the development error detection feature.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	False
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.4.4 Parameter MPU_UserModeSupport

Enables the development error detection feature.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	False
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Chapter 6

Module Documentation

6.1 Interrupt Controller IP

6.1.1 Detailed Description

Data Structures

- struct IntCtrl_Ip_IrqConfigType

  Structure storing the state and priority configuration for an interrupt request. More...
- struct IntCtrl_Ip_CtrlConfigType

  Structure storing the list of state configurations for all configured interrupts. More...

Types Reference

- typedef void(∗ IntCtrl_Ip_IrqHandlerType) (void)

  Interrupt handler type.

Enum Reference

- enum IntCtrl_Ip_StatusType

  Enumeration listing the possible error codes returned by IntCtrl_Ip API.

Function Reference

- IntCtrl_Ip_StatusType IntCtrl_Ip_Init (const IntCtrl_Ip_CtrlConfigType ∗pIntCtrlCtrlConfig)

  Initializes the configured interrupts at interrupt controller level.
- void IntCtrl_Ip_InstallHandler (IRQn_Type eIrqNumber, const IntCtrl_Ip_IrqHandlerType pfNewHandler, IntCtrl_Ip_IrqHandlerType ∗const pfOldHandler)

  Installs a handler for an IRQ.
- void IntCtrl_Ip_EnableIrq (IRQn_Type eIrqNumber)

  Enables an interrupt request.
- void IntCtrl_Ip_DisableIrq (IRQn_Type eIrqNumber)

  Disables an interrupt request.
- void IntCtrl_Ip_SetPriority (IRQn_Type eIrqNumber, uint8 u8Priority)

  Sets the priority for an interrupt request.
- uint8 IntCtrl_Ip_GetPriority (IRQn_Type eIrqNumber)

Gets the priority for an interrupt request.

- void IntCtrl_Ip_ClearPending (IRQn_Type eIrqNumber)

  Clears the pending flag for an interrupt request.

6.1.2 Data Structure Documentation

6.1.2.1 struct IntCtrl_Ip_IrqConfigType

Structure storing the state and priority configuration for an interrupt request.

Definition at line 85 of file IntCtrl_Ip_TypesDef.h.

Data Fields

- IRQn_Type eIrqNumber

  Interrupt number.

- boolean bIrqEnabled

  Interrupt state (enabled/disabled)

- uint8 u8IrqPriority

  Interrupt priority.

- IntCtrl_Ip_IrqHandlerType pfHandler

  Interrupt handler.

Field Documentation

eIrqNumber

```
IRQn_Type eIrqNumber
```

Interrupt number.

Definition at line 88 of file IntCtrl_Ip_TypesDef.h.

bIrqEnabled

```
boolean bIrqEnabled
```

Interrupt state (enabled/disabled)

Definition at line 90 of file IntCtrl_Ip_TypesDef.h.

u8IrqPriority

```
uint8 u8IrqPriority
```

Interrupt priority.

Definition at line 92 of file IntCtrl_Ip_TypesDef.h.

pfHandler

`IntCtrl_Ip_IrqHandlerType pfHandler`

Interrupt handler.

Definition at line 94 of file IntCtrl_Ip_TypesDef.h.

6.1.2.2 struct IntCtrl_Ip_CtrlConfigType

Structure storing the list of state configurations for all configured interrupts.

Definition at line 101 of file IntCtrl_Ip_TypesDef.h.

Data Fields

- uint32 u32ConfigIrqCount

  Number of configured interrupts.
- const IntCtrl_Ip_IrqConfigType * aIrqConfig

  List of interrupts configurations.

Field Documentation

u32ConfigIrqCount

`uint32 u32ConfigIrqCount`

Number of configured interrupts.

Definition at line 104 of file IntCtrl_Ip_TypesDef.h.

aIrqConfig

`const IntCtrl_Ip_IrqConfigType* aIrqConfig`

List of interrupts configurations.

Definition at line 110 of file IntCtrl_Ip_TypesDef.h.

6.1.3 Types Reference

6.1.3.1 IntCtrl_Ip_IrqHandlerType

```
typedef void(* IntCtrl_Ip_IrqHandlerType) (void)
```

Interrupt handler type.

Definition at line 78 of file IntCtrl_Ip_TypesDef.h.

6.1.4 Enum Reference

6.1.4.1 IntCtrl_Ip_StatusType

```
enum IntCtrl_Ip_StatusType
```

Enumeration listing the possible error codes returned by IntCtrl_Ip API.

Enumerator

INTCTRL_IP_STATUS_SUCCESS	Status SUCCESS.
INTCTRL_IP_STATUS_ERROR	Status ERROR.

Definition at line 117 of file IntCtrl_Ip_TypesDef.h.

6.1.5 Function Reference

6.1.5.1 IntCtrl_Ip_Init()

```
IntCtrl_Ip_StatusType IntCtrl_Ip_Init (
            const IntCtrl_Ip_CtrlConfigType * pIntCtrlCtrlConfig )
```

Initializes the configured interrupts at interrupt controller level.

This function is non-reentrant and initializes the interrupts.

Parameters

in	*pIntCtrlCtrlConfig*	pointer to configuration structure for interrupts.

Returns

IntCtrl_Ip_StatusType: error code.

6.1.5.2 IntCtrl_Ip_InstallHandler()

```
void IntCtrl_Ip_InstallHandler (
            IRQn_Type eIrqNumber,
            const IntCtrl_Ip_IrqHandlerType pfNewHandler,
            IntCtrl_Ip_IrqHandlerType *const pfOldHandler )
```

Installs a handler for an IRQ.

This function is non-reentrant; it installs an new ISR for an interrupt line.

Note

This function works only when the interrupt vector table resides in RAM.

Parameters

in	*eIrqNumber*	interrupt number.
in	*pfNewHandler*	function pointer for the new handler.
out	*pfOldHandler*	stores the address of the old interrupt handler.

Returns

void.

6.1.5.3 IntCtrl_Ip_EnableIrq()

```
void IntCtrl_Ip_EnableIrq (
             IRQn_Type eIrqNumber )
```

Enables an interrupt request.

This function is non-reentrant; it enables the interrupt request at interrupt controller level.

Parameters

in	*eIrqNumber*	interrupt number to be enabled.

Returns

void.

6.1.5.4 IntCtrl_Ip_DisableIrq()

```
void IntCtrl_Ip_DisableIrq (
             IRQn_Type eIrqNumber )
```

Disables an interrupt request.

This function is non-reentrant; it disables the interrupt request at interrupt controller level.

Parameters

in	*eIrqNumber*	interrupt number to be disabled.

Returns

void.

6.1.5.5 IntCtrl_Ip_SetPriority()

```
void IntCtrl_Ip_SetPriority (
             IRQn_Type eIrqNumber,
             uint8 u8Priority )
```

Sets the priority for an interrupt request.

This function is non-reentrant; it sets the priority for the interrupt request.

Parameters

in	*eIrqNumber*	interrupt number for which the priority is set.
in	*u8Priority*	the priority to be set.

Returns

void.

6.1.5.6 IntCtrl_Ip_GetPriority()

```
uint8 IntCtrl_Ip_GetPriority (
            IRQn_Type eIrqNumber )
```

Gets the priority for an interrupt request.

This function is non-reentrant; it retrieves the priority for the interrupt request.

Parameters

in	*eIrqNumber*	interrupt number for which the priority is set.

Returns

uint8: the priority of the interrupt.

6.1.5.7 IntCtrl_Ip_ClearPending()

```
void IntCtrl_Ip_ClearPending (
            IRQn_Type eIrqNumber )
```

Clears the pending flag for an interrupt request.

This function is reentrant; it clears the pending flag for the interrupt request.

Parameters

in	*eIrqNumber*	interrupt number for which the pending flag is cleared.

Returns

void.

6.2 MPU IPV Driver

6.2.1 Detailed Description

Data Structures

- struct Mpu_Ip_RegionConfigType

  Configuration structure containing the region configuration. More...
- struct Mpu_Ip_ConfigType

  IP configuration structure. More...
- struct Mpu_Ip_ErrorDetailsType

  Structure used to retrieve violation details. More...

Enum Reference

- enum Mpu_Ip_SupervisorAccessModeType

  Enumeration listing access permissions in supervisor mode.
- enum Mpu_Ip_UserAccessModeType

  Enumeration listing access permissions in user mode.
- enum Mpu_Ip_MasterType

  Enumeration listing masters.
- enum Mpu_Ip_ErrorAttributesType

  Structure used to retrieve error attributes details.
- enum Mpu_Ip_AccessType

  Structure used to retrieve error attributes details.

Function Reference

- void Mpu_Ip_Init (const Mpu_Ip_ConfigType *pConfig)

  Initializes the Memory Protection Unit general parameters and region configurations.
- void Mpu_Ip_SetRegionConfig (uint8 u8RegionNum, const Mpu_Ip_RegionConfigType *const pUser↩ ConfigPtr)

  Configures the region selected by u8RegionNum with the data from pUserConfigPtr.
- void Mpu_Ip_Deinit (void)

  Disables the module and resets all region configurations.
- void Mpu_Ip_EnableRegion (uint8 u8RegionNum, boolean bEnable)

  Enables or disabled a specific region.
- void Mpu_Ip_SetAccessMode (uint8 u8RegionNum, Mpu_Ip_MasterType eMaster, Mpu_Ip_SupervisorAccessModeType eSupervisorMode, Mpu_Ip_UserAccessModeType eUserMode)

  Modify the access mode for a master to a specific region.
- boolean Mpu_Ip_GetErrorDetails (Mpu_Ip_ErrorDetailsType *pErrorDetails)

  Retrieve error details.
- void Mpu_Ip_Init_Privileged (const Mpu_Ip_ConfigType *pConfig)

  Initializes the MPU instance and memory regions configured.

- void Mpu_Ip_SetRegionConfig_Privileged (const uint8 regionNumber, const Mpu_Ip_RegionConfigType *const pUserConfigPtr)

  Configures the specified region number using the input region configuration.
- void Mpu_Ip_Deinit_Privileged (void)

  Deinitialize MPU instance.
- void Mpu_Ip_EnableRegion_Privileged (uint8 u8RegionNum, boolean bEnable)

  Enable or disable region configuration.
- void Mpu_Ip_SetAccessMode_Privileged (uint8 u8RegionNum, Mpu_Ip_MasterType eMaster, Mpu_Ip_SupervisorAcces eSupervisorMode, Mpu_Ip_UserAccessModeType eUserMode)

  Modify the access mode for a master to a specific region.
- boolean Mpu_Ip_GetErrorDetails_Privileged (Mpu_Ip_ErrorDetailsType *pErrorDetails)

  Retrieves error details such as address and error type.

6.2.2 Data Structure Documentation

6.2.2.1 struct Mpu_Ip_RegionConfigType

Configuration structure containing the region configuration.

Definition at line 101 of file Mpu_Ip_TypesDef.h.

Data Fields

uint32	u32StartAddr	Memory region start address - Word 0
uint32	u32EndAddr	Memory region end address - Word 1
uint32	u32Word2	Access permission for region - Word 2
uint32	u32Pid	Process Identifier - Word 3
uint32	u32PidMask	Process Identifier Mask - Word 3

6.2.2.2 struct Mpu_Ip_ConfigType

IP configuration structure.

Definition at line 113 of file Mpu_Ip_TypesDef.h.

Data Fields

uint8	u8RegionCnt	Region Count
const Mpu_Ip_RegionConfigType *	pRegionConfigArr	Region configuration array
const uint8 *	pRegionNumberArr	Region Number array

6.2.2.3 struct Mpu_Ip_ErrorDetailsType

Structure used to retrieve violation details.

Definition at line 144 of file Mpu_Ip_TypesDef.h.

Data Fields

uint32	u32Address	Violation address
uint16	u16EACD	Error Access Control Detail
Mpu_Ip_MasterType	eMaster	Violation master
Mpu_Ip_ErrorAttributesType	eErrorAttribute	Type of Attribute violation
Mpu_Ip_AccessType	eErrorAccess	Type of Access violation

6.2.3 Enum Reference

6.2.3.1 Mpu_Ip_SupervisorAccessModeType

`enum Mpu_Ip_SupervisorAccessModeType`

Enumeration listing access permissions in supervisor mode.

Enumerator

MPU_SUPERVISOR_MODE_RWX	0b00U : rwx|
MPU_SUPERVISOR_MODE_RX	0b01U : r-x|
MPU_SUPERVISOR_MODE_RW	0b10U : rw-|
MPU_SUPERVISOR_MODE_AS_USER_MODE	0b11U : —|

Definition at line 62 of file Mpu_Ip_TypesDef.h.

6.2.3.2 Mpu_Ip_UserAccessModeType

`enum Mpu_Ip_UserAccessModeType`

Enumeration listing access permissions in user mode.

Enumerator

MPU_USER_MODE_NONE	0b000U : —|
MPU_USER_MODE_X	0b001U : –x|
MPU_USER_MODE_W	0b010U : -w-|
MPU_USER_MODE_WX	0b011U : -wx|
MPU_USER_MODE_R	0b100U : r–|
MPU_USER_MODE_RX	0b101U : r-x|
MPU_USER_MODE_RW	0b110U : rw-|
MPU_USER_MODE_RWX	0b111U : rwx|

Definition at line 74 of file Mpu_Ip_TypesDef.h.

6.2.3.3 Mpu_Ip_MasterType

enum Mpu_Ip_MasterType

Enumeration listing masters.

Definition at line 88 of file Mpu_Ip_TypesDef.h.

6.2.3.4 Mpu_Ip_ErrorAttributesType

enum Mpu_Ip_ErrorAttributesType

Structure used to retrieve error attributes details.

Definition at line 123 of file Mpu_Ip_TypesDef.h.

6.2.3.5 Mpu_Ip_AccessType

enum Mpu_Ip_AccessType

Structure used to retrieve error attributes details.

Definition at line 135 of file Mpu_Ip_TypesDef.h.

6.2.4 Function Reference

6.2.4.1 Mpu_Ip_Init()

```
void Mpu_Ip_Init (
            const Mpu_Ip_ConfigType * pConfig )
```

Initializes the Memory Protection Unit general parameters and region configurations.

This function is non-reentrant

Parameters

in	*pConfig*	pointer to configuration structure for MPU.

Returns

void

Precondition

None

6.2.4.2 Mpu_Ip_SetRegionConfig()

```
void Mpu_Ip_SetRegionConfig (
            uint8 u8RegionNum,
            const Mpu_Ip_RegionConfigType *const pUserConfigPtr )
```

Configures the region selected by u8RegionNum with the data from pUserConfigPtr.

This function is Reentrant

Parameters

in	*u8RegionNum*	Region to be modified .
in	*pUserConfigPtr*	pointer to the region configuration structure for MPU.

Returns

void

Precondition

6.2.4.3 Mpu_Ip_Deinit()

```
void Mpu_Ip_Deinit (
             void  )
```

Disables the module and resets all region configurations.

This function is Reentrant

Returns

Void

Precondition

None

6.2.4.4 Mpu_Ip_EnableRegion()

```
void Mpu_Ip_EnableRegion (
             uint8 u8RegionNum,
             boolean bEnable )
```

Enables or disabled a specific region.

This function is Reentrant

Parameters

in	*u8Region*	: Region to be modified
in	*bEnable*	: Specifies wheter the region is enabled or disabled

Returns

void

Precondition

None

6.2.4.5 Mpu_Ip_SetAccessMode()

```
void Mpu_Ip_SetAccessMode (
            uint8 u8RegionNum,
            Mpu_Ip_MasterType eMaster,
            Mpu_Ip_SupervisorAccessModeType eSupervisorMode,
            Mpu_Ip_UserAccessModeType eUserMode )
```

Modify the access mode for a master to a specific region.

This function is Reentrant

Parameters

in	*u8RegionNum*	: Region to be modified
in	*eMaster*	: Master to be modified
in	*eSupervisorMode*	: Specifies the new mode access in supervisor mode
in	*eUserMode*	: Specifies the new mode access in user mode

Returns

void

Precondition

None

6.2.4.6 Mpu_Ip_GetErrorDetails()

```
boolean Mpu_Ip_GetErrorDetails (
            Mpu_Ip_ErrorDetailsType * pErrorDetails )
```

Retrieve error details.

This function is Reentrant

Parameters

out	*pErrorDetails*	: Storage where the data will be saved

Returns

boolean - TRUE if an error was present, FALSE otherwise

Precondition

None

6.2.4.7 Mpu_Ip_Init_Privileged()

```
void Mpu_Ip_Init_Privileged (
            const Mpu_Ip_ConfigType * pConfig )
```

Initializes the MPU instance and memory regions configured.

6.2.4.8 Mpu_Ip_SetRegionConfig_Privileged()

```
void Mpu_Ip_SetRegionConfig_Privileged (
             const uint8 regionNumber,
             const Mpu_Ip_RegionConfigType *const pUserConfigPtr )
```

Configures the specified region number using the input region configuration.

6.2.4.9 Mpu_Ip_Deinit_Privileged()

```
void Mpu_Ip_Deinit_Privileged (
             void  )
```

Deinitialize MPU instance.

6.2.4.10 Mpu_Ip_EnableRegion_Privileged()

```
void Mpu_Ip_EnableRegion_Privileged (
             uint8 u8RegionNum,
             boolean bEnable )
```

Enable or disable region configuration.

6.2.4.11 Mpu_Ip_SetAccessMode_Privileged()

```
void Mpu_Ip_SetAccessMode_Privileged (
             uint8 u8RegionNum,
             Mpu_Ip_MasterType eMaster,
             Mpu_Ip_SupervisorAccessModeType eSupervisorMode,
             Mpu_Ip_UserAccessModeType eUserMode )
```

Modify the access mode for a master to a specific region.

6.2.4.12 Mpu_Ip_GetErrorDetails_Privileged()

```
boolean Mpu_Ip_GetErrorDetails_Privileged (
             Mpu_Ip_ErrorDetailsType * pErrorDetails )
```

Retrieves error details such as address and error type.

6.3 Platform

6.3.1 Detailed Description

Modules

- Interrupt Controller IP
- System IP

Data Structures

- struct Platform_ConfigType

  Configuration structure for PLATFORM CDD. More...

Macros

- #define PLATFORM_UNINIT

  PLATFORM driver states.
- #define PLATFORM_INIT

  PLATFORM driver states.
- #define PLATFORM_E_PARAM_POINTER

  All API's having pointers as parameters shall return this error if called with with a NULL value.
- #define PLATFORM_E_PARAM_OUT_OF_RANGE

  Error returned for parameters out of range.
- #define PLATFORM_E_UNINIT

  API service used without module initialization shall return this error.
- #define PLATFORM_E_PARAM_CONFIG

  If DET error reporting is enabled, the PLATFORM will check upon each API call if the requested resource is configured to be available on the current core, and in case of error will return PLATFORM_E_PARAM_CONFIG.
- #define PLATFORM_E_PARAM_CHANNEL

  API service called with wrong parameter of Channel.
- #define PLATFORM_E_PARAM_INSTANCE

  API service called with wrong parameter of Instance.
- #define PLATFORM_INIT_ID

  Service ID of Platform_Init function.
- #define PLATFORM_SET_IRQ_ID

  Service ID of Platform_SetIrq function.
- #define PLATFORM_SET_IRQ_PRIO_ID

  Service ID of Platform_SetIrqPriority function.
- #define PLATFORM_GET_IRQ_PRIO_ID

  Service ID of Platform_GetIrqPriority function.
- #define PLATFORM_INSTALL_HANDLER_ID

  Service ID of Platform_InstallIrqHandler function.

Types Reference

- typedef IntCtrl_Ip_IrqHandlerType Platform_IrqHandlerType

  Interrupt handler type definition for PLATFORM CDD.

Function Reference

- void Platform_Init (const Platform_ConfigType *pConfig)

  Initializes the paltform settings based on user configuration.

- Std_ReturnType Platform_SetIrq (IRQn_Type eIrqNumber, boolean bEnable)

  Configures (enables/disables) an interrupt request.

- Std_ReturnType Platform_SetIrqPriority (IRQn_Type eIrqNumber, uint8 u8Priority)

  Configures the priority of an interrupt request.

- Std_ReturnType Platform_GetIrqPriority (IRQn_Type eIrqNumber, uint8 *u8Priority)

  Returns the priority of an interrupt request.

- Std_ReturnType Platform_InstallIrqHandler (IRQn_Type eIrqNumber, const Platform_IrqHandlerType pfNewHandler, Platform_IrqHandlerType *const pfOldHandler)

  Installs a new handler for an interrupt request.

- void Platform_Mpu_SetRegionConfig (uint8 u8RegionNum, const Platform_Mpu_RegionConfigType *const pUserConfigPtr)

  Configures the region selected by u8RegionNum with the data from pUserConfigPtr.

- void Platform_Mpu_EnableRegion (uint8 u8RegionNum, boolean bEnable)

  Enables or disabled a specific region.

- void Platform_Mpu_SetAccessMode (uint8 u8RegionNum, Platform_Mpu_MasterType eMaster, Platform_Mpu_SupervisorAccessModeType eSupervisorMode, Platform_Mpu_UserAccessModeType eUserMode)

  Modify the access mode for a master to a specific region.

- Std_ReturnType Platform_Mpu_GetErrorDetails (Platform_Mpu_ErrorDetailsType *pErrorDetails)

  Retrieve error details.

6.3.2 Data Structure Documentation

6.3.2.1 struct Platform_ConfigType

Configuration structure for PLATFORM CDD.

Definition at line 182 of file Platform_TypesDef.h.

Data Fields

- const Platform_Ipw_ConfigType * pIpwConfig

  Reference to IPW structure.

Field Documentation

pIpwConfig

```
const Platform_Ipw_ConfigType* pIpwConfig
```

Reference to IPW structure.

Definition at line 185 of file Platform_TypesDef.h.

6.3.3 Macro Definition Documentation

6.3.3.1 PLATFORM_UNINIT

```
#define PLATFORM_UNINIT
```

PLATFORM driver states.

The state PLATFORM_UNINIT means that the PLATFORM module has not been initialized.

Definition at line 92 of file Platform_TypesDef.h.

6.3.3.2 PLATFORM_INIT

```
#define PLATFORM_INIT
```

PLATFORM driver states.

The PLATFORM_INIT state indicates that the PLATFORM driver has been initialized.

Definition at line 101 of file Platform_TypesDef.h.

6.3.3.3 PLATFORM_E_PARAM_POINTER

```
#define PLATFORM_E_PARAM_POINTER
```

All API's having pointers as parameters shall return this error if called with with a NULL value.

Definition at line 108 of file Platform_TypesDef.h.

6.3.3.4 PLATFORM_E_PARAM_OUT_OF_RANGE

```
#define PLATFORM_E_PARAM_OUT_OF_RANGE
```

Error returned for parameters out of range.

Definition at line 114 of file Platform_TypesDef.h.

6.3.3.5 PLATFORM_E_UNINIT

```
#define PLATFORM_E_UNINIT
```

API service used without module initialization shall return this error.

Definition at line 120 of file Platform_TypesDef.h.

6.3.3.6 PLATFORM_E_PARAM_CONFIG

```
#define PLATFORM_E_PARAM_CONFIG
```

If DET error reporting is enabled, the PLATFORM will check upon each API call if the requested resource is configured to be available on the current core, and in case of error will return PLATFORM_E_PARAM_CONFIG.

Definition at line 128 of file Platform_TypesDef.h.

6.3.3.7 PLATFORM_E_PARAM_CHANNEL

```
#define PLATFORM_E_PARAM_CHANNEL
```

API service called with wrong parameter of Channel.

Definition at line 134 of file Platform_TypesDef.h.

6.3.3.8 PLATFORM_E_PARAM_INSTANCE

```
#define PLATFORM_E_PARAM_INSTANCE
```

API service called with wrong parameter of Instance.

Definition at line 140 of file Platform_TypesDef.h.

6.3.3.9 PLATFORM_INIT_ID

```
#define PLATFORM_INIT_ID
```

Service ID of Platform_Init function.

Parameter used when raising an error/exception

Definition at line 146 of file Platform_TypesDef.h.

6.3.3.10 PLATFORM_SET_IRQ_ID

```
#define PLATFORM_SET_IRQ_ID
```

Service ID of Platform_SetIrq function.

Parameter used when raising an error/exception

Definition at line 152 of file Platform_TypesDef.h.

6.3.3.11 PLATFORM_SET_IRQ_PRIO_ID

```
#define PLATFORM_SET_IRQ_PRIO_ID
```

Service ID of Platform_SetIrqPriority function.

Parameter used when raising an error/exception

Definition at line 158 of file Platform_TypesDef.h.

6.3.3.12 PLATFORM_GET_IRQ_PRIO_ID

```
#define PLATFORM_GET_IRQ_PRIO_ID
```

Service ID of Platform_GetIrqPriority function.

Parameter used when raising an error/exception

Definition at line 164 of file Platform_TypesDef.h.

6.3.3.13 PLATFORM_INSTALL_HANDLER_ID

```
#define PLATFORM_INSTALL_HANDLER_ID
```

Service ID of Platform_InstallIrqHandler function.

Parameter used when raising an error/exception

Definition at line 170 of file Platform_TypesDef.h.

6.3.4 Types Reference

6.3.4.1 Platform_IrqHandlerType

```
typedef IntCtrl_Ip_IrqHandlerType Platform_IrqHandlerType
```

Interrupt handler type definition for PLATFORM CDD.

Definition at line 194 of file Platform_TypesDef.h.

6.3.5 Function Reference

6.3.5.1 Platform_Init()

```
void Platform_Init (
            const Platform_ConfigType * pConfig )
```

Initializes the paltform settings based on user configuration.

This function is non-reentrant; it initializes the interrupts, interrupt monitors (if available), as well as other platform specific settings as defined for each SoC.

Parameters

in	*pConfig*	pointer to platform configuration structure.

Returns

void

6.3.5.2 Platform_SetIrq()

```
Std_ReturnType Platform_SetIrq (
            IRQn_Type eIrqNumber,
```

```
            boolean bEnable )
```

Configures (enables/disables) an interrupt request.

This function is non-reentrant; it enables/disables the selected interrupt.

Parameters

in	*eIrqNumber*	interrupt to be configured.
in	*bEnable*	TRUE - enable interrupt, FALSE - disable interrupt.

Returns

Std_ReturnType: E_OK/E_NOT_OK; specific errors are reported through DET.

6.3.5.3 Platform_SetIrqPriority()

```
Std_ReturnType Platform_SetIrqPriority (
            IRQn_Type eIrqNumber,
            uint8 u8Priority )
```

Configures the priority of an interrupt request.

This function is non-reentrant; it sets the priority for the selected interrupt.

Parameters

in	*eIrqNumber*	interrupt number for which priority is configured.
in	*u8Priority*	desired priority of the interrupt.

Returns

Std_ReturnType: E_OK/E_NOT_OK; specific errors are reported through DET.

6.3.5.4 Platform_GetIrqPriority()

```
Std_ReturnType Platform_GetIrqPriority (
            IRQn_Type eIrqNumber,
            uint8 * u8Priority )
```

Returns the priority of an interrupt request.

This function is non-reentrant; it retrieves the current priority of the selected interrupt.

Parameters

in	*eIrqNumber*	interrupt number for which priority is returned.
out	*u8Priority*	output parameter storing the priority of the interrupt.

Returns

Std_ReturnType: E_OK/E_NOT_OK; specific errors are reported through DET.

6.3.5.5 Platform_InstallIrqHandler()

```
Std_ReturnType Platform_InstallIrqHandler (
            IRQn_Type eIrqNumber,
            const Platform_IrqHandlerType pfNewHandler,
            Platform_IrqHandlerType *const pfOldHandler )
```

Installs a new handler for an interrupt request.

This function is non-reentrant; it replaces the current interrupt handler for the selected interrupt with the new function provided as the second parameter. The address of the old handler can be optionally stored in the third parameter.

Parameters

in	*eIrqNumber*	interrupt number for which priority is returned.
in	*pfNewHandler*	function pointer for the new handler.
out	*pfOldHandler*	function pointer that will store the address of the old handler

Note

- this parameter can be passed as NULL if not needed.

Returns

pfOldHandler: E_OK/E_NOT_OK; specific errors are reported through DET.

6.3.5.6 Platform_Mpu_SetRegionConfig()

```
void Platform_Mpu_SetRegionConfig (
            uint8 u8RegionNum,
            const Platform_Mpu_RegionConfigType *const pUserConfigPtr )
```

Configures the region selected by u8RegionNum with the data from pUserConfigPtr.

This function is Reentrant

Parameters

in	*u8RegionNum*	region number
in	*pUserConfigPtr*	pointer to the region configuration structure for MPU.

Returns

void

Precondition

6.3.5.7 Platform_Mpu_EnableRegion()

```
void Platform_Mpu_EnableRegion (
            uint8 u8RegionNum,
            boolean bEnable )
```

Enables or disabled a specific region.

This function is Reentrant

Parameters

in	*u8Region*	: Region to be modified
in	*bEnable*	: Specifies whether the region is enabled or disabled

Returns

void

Precondition

6.3.5.8 Platform_Mpu_SetAccessMode()

```
void Platform_Mpu_SetAccessMode (
            uint8 u8RegionNum,
            Platform_Mpu_MasterType eMaster,
            Platform_Mpu_SupervisorAccessModeType eSupervisorMode,
            Platform_Mpu_UserAccessModeType eUserMode )
```

Modify the access mode for a master to a specific region.

This function is Reentrant

Parameters

in	*u8RegionNum*	: Region to be modified
in	*eSupervisorMode*	: Specifies the new mode access in supervisor mode
in	*eUserMode*	: Specifies the new mode access in user mode

Returns

void

Precondition

6.3.5.9 Platform_Mpu_GetErrorDetails()

```
Std_ReturnType Platform_Mpu_GetErrorDetails (
            Platform_Mpu_ErrorDetailsType * pErrorDetails )
```

Retrieve error details.

This function is Reentrant

Parameters

out	*pErrorDetails*	: Storage where the data will be saved

Returns

Std_ReturnType

Return values

E_OK	if an error was present
E_NOT_OK	otherwise

Precondition

6.4 System IP

6.4.1 Detailed Description

Function Reference

- void System_Ip_ConfigIrq (System_Ip_IrqType eIrq, boolean bEnable)

  Enables/disables core-related interrupt exceptions.

6.4.2 Function Reference

6.4.2.1 System_Ip_ConfigIrq()

```
void System_Ip_ConfigIrq (
            System_Ip_IrqType eIrq,
            boolean bEnable )
```

Enables/disables core-related interrupt exceptions.

This function is non-reentrant and configures core-related interrupt exceptions, as defined per each platform.

Parameters

in	*eIrq*	core-related interrupt event.
in	*bEnable*	FALSE - disable interrupt, TRUE - enable interrupt.

Returns

void

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.
In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.
Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.
Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.
NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at http://www.nxp.com/profile/terms, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.
Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.
NXP has a Product Security Incident Response Team (PSIRT) (reachable at mailto:PSIRT@nxp.com) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.